RSCL — Algorithm 19: Voice Transcript Normalisation and Spoken-Confirmation Protocol for Corrected Transcripts

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)
Source files described (content hash and state at document build time):

`hu-telepresence-gazebo/festival/hu_live_server.py`	`sha256 2416370e93dc`	2026-11-11 06:38 UTC	clean @ 10ee643
`hu-telepresence-gazebo/festival/hu_live_dashboard.html`	`sha256 d1e2cf6aabae`	2026-11-12 01:23 UTC	clean @ 05172be

1. Purpose and context

Browser speech recognition mishears the aircraft names (Ra as "rock" or "raw", Horus as "chorus" or "horror") and accented words ("yella" for *yalla*, "old" for *all*). Silently rewriting a transcript and then executing it would let a recogniser error choose the aircraft. This algorithm repairs a fixed set of known confusions but never executes a repaired command without a second, separate confirmation, with one deliberate exception: emergency phrases (Algorithm 2), where delay is the larger risk.

2. Inputs, outputs, state

- **Inputs:** raw transcript (1..300 chars) on `/api/hu/voice-auto`; a clarification id on `/api/hu/voice-accept`; `/api/hu/cancel`; dashboard review path `/api/hu/command` with `voice_reviewed`.
- **State:** server: `voice_pending`, at most one entry {utterance (corrected), heard (raw), at}, id of 12 hex characters; the command table `pending` of Algorithm 1. Client (hands-free dashboard): one `pending` object of kind *alias* or *hu* with a 45 s timer.
- **Outputs:** decision records with band VOICE_REVIEW, VOICE_CLARIFY, EMERGENCY, or the bands of Algorithms 1/3; `original_transcript` is kept in every record.

3. Algorithm

Algorithm 19.1 Normalisation and clarification

Source: festival/hu_live_server.py: VOICE_ALIASES, normalize_utterance, Service.voice_auto, Service.voice_accept, Service.cancel, Service.command

```
function NORMALIZE(raw, source)
  u ← raw
  if source = voice then
    for (wrong → right) in VOICE_ALIASES do u ← replace whole-word, case-insensitive
    u ← replace whole word "old" by "all" only when followed by [the] [drone(s)] + one of
         take off|takeoff|land|stop|hover|form|perform|turn|rotate|spin
  return u
procedure VOICE-AUTO(raw)                                  // hands-free route
  s ← NORMALIZE(raw, voice)
  if EMERGENCY-KEY(s) ≠ None then return COMMAND(s, voice, reviewed, original = raw)   // Alg. 2, immediate
  if s ≠ raw then
    voice_pending ← { new id: (s, raw, now) }              // clears any earlier clarification
    return VOICE_CLARIFY (no dispatch; operator must say "yes")
  return COMMAND(raw, voice, reviewed, original = raw)  // normal pipeline (Alg. 2 → 3 → 1)
procedure VOICE-ACCEPT(id)
  c ← voice_pending.pop(id);  if c = None then refuse "missing or already used"
  if now − c.at > 75 s then refuse "expired; repeat the command"
```

```
  return COMMAND(c.utterance, voice, reviewed, original = c.heard)
procedure COMMAND(u, source, reviewed)                  // first lines of Service.command
  if source = voice and not reviewed then return VOICE_REVIEW (no inference, no dispatch)
  continue with Algorithm 2, then 3, then 1 on NORMALIZE(u, source)
```

4. Flowchart and state diagram

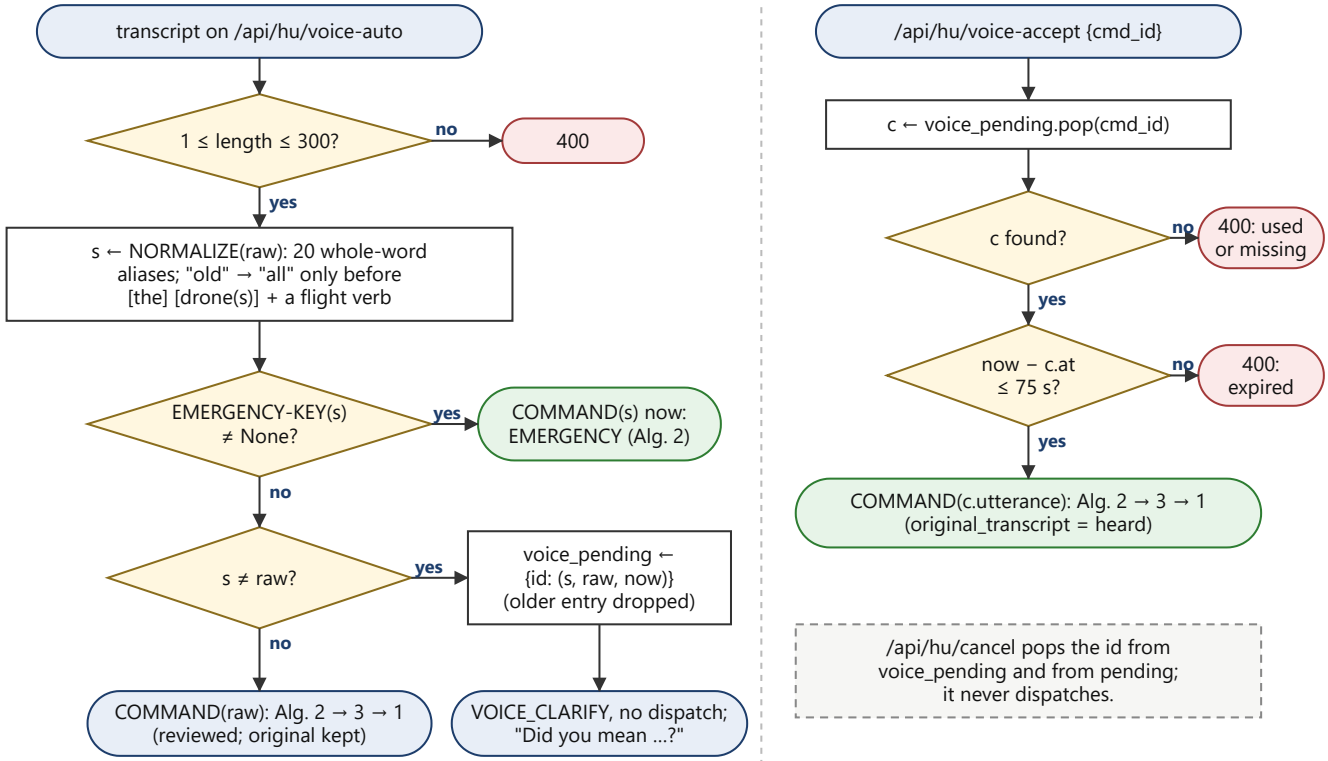


Figure 19.1. Flowcharts of `Service.voice_auto` (left) and `Service.voice_accept` (right). The entry is popped before the age test, so an expired id is also consumed. A command produced by an accepted correction still passes through all of Algorithms 2, 3 and 1; acceptance of the correction is not authorisation of the command.

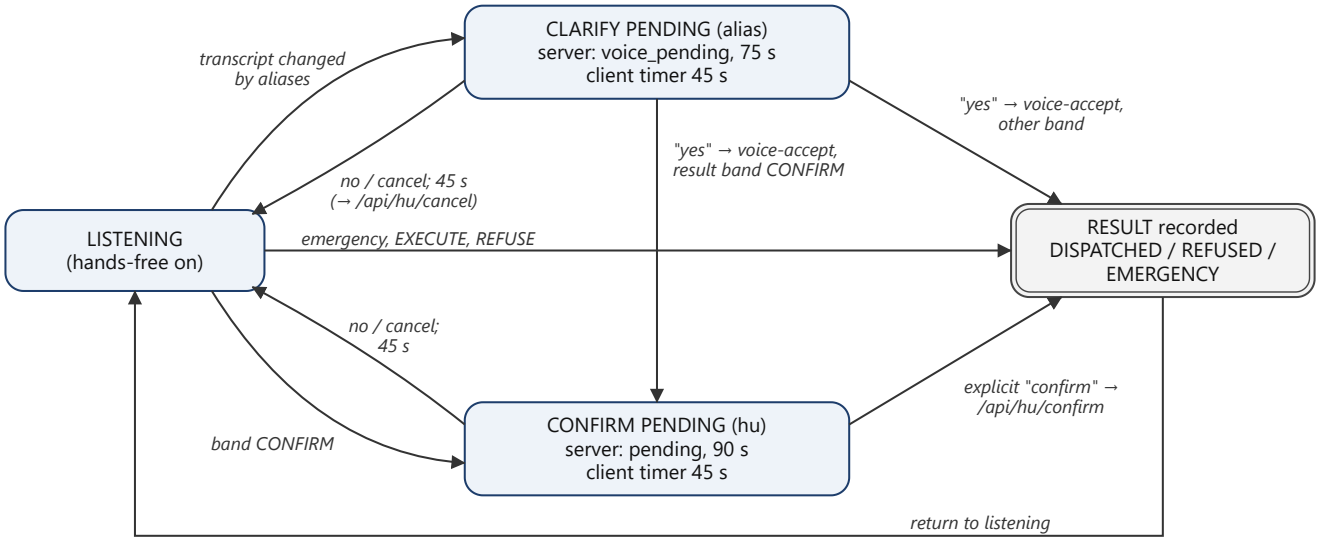


Figure 19.2. Dialogue state machine of the hands-free voice front end (server: `voice_pending`, `pending`; client: `setPending`, `recognizedYes`, `recognizedCancel` in `hu_live_dashboard.html`). In CLARIFY, "yes", "yeah", "correct", "that's right", "yes please" or an explicit confirm phrase accepts; in CONFIRM only an explicit phrase ("confirm", "I confirm", "affirmative", ...) authorises, and bare "yes" is not accepted. Any other utterance in either pending state re-prompts (it is not sent), except an emergency phrase, which cancels the pending entry and is dispatched at once. The client timeout (45 s) is shorter than the server lifetimes.

5. Parameters

Item	Value
Aliases → Ra	rock, prop, roland, raw, rah
Aliases → Horus	taurus, horace, chorus, horis, horror, horrors
Aliases → Anubis / Thoth	anubus, "a nubis" / toth, thought, thoss
Aliases → yalla	yella, yallow, yellow, yala
Clarification lifetime	75 s, single use, at most one outstanding
Command confirmation lifetime (separate)	90 s (Algorithm 1/3)

6. Complexity

$O(|A|\cdot|u|)$ for $|A| = 20$ alias substitutions plus one contextual regular expression.

7. Properties

Guaranteed

- A transcript changed by normalisation is never dispatched on the same request (except emergency phrases); it requires a separate accept carrying a server-held id within 75 s.
- Only one clarification can be pending; a newer one replaces it, a used one cannot be replayed.
- "old" is rewritten only in the listed verb contexts; other uses are untouched.
- Unreviewed voice input on the dashboard route never reaches model inference.
- Emergency phrases, including alias-corrected ones ("yella land"), act immediately.

Not guaranteed

- Common words in the alias list (e.g. *rock*, *thought*, *yellow*, *prop*) are rewritten wherever they occur; this is safe only because the result must be confirmed (or is an emergency).

- An uncorrected but misrecognised transcript (e.g. a wrong but valid name) proceeds to normal gating without extra confirmation; protection then comes from Algorithms 1/3 (CONFIRM band for mission proposals; EXECUTE allowed for confident HU primitives).
- "Yes" and "confirm" are recognised in the browser by exact phrase lists (Figure 19.2); robustness to recogniser errors is not measured. A misrecognised "yes" can accept a correction, but cannot by itself authorise a CONFIRM-band command.
- Only the first recognition alternative is used and its confidence is ignored, although the dashboard requests three alternatives.
- The 45 s timeout is client-side. If the page is closed, server entries remain until their 75 s / 90 s lifetimes are exceeded, after which an accept or confirm is refused.

8. Testing and measured results

- `acceptance/tests/test_v9.py::test_voice_aliases_suggest_review`; `test_missions.py::test_voice_confirmation_mission`.
- Live command matrix (`run_20261111T052914Z`): "old take off", "horror take off", "raw take off" → VOICE_CLARIFY, no motion. Fault scenario: voice "yella land" → EMERGENCY, all grounded in 6.39 s.
- Recogniser error rates and alias coverage: **not measured here**.

9. Comparison with alternatives and related work

Spoken dialogue systems handle recognition errors with explicit or implicit confirmation, confidence-dependent strategies, or belief tracking; aviation radiotelephony uses read-back. These are compared with the protocol of this document.

- **Explicit and implicit verification** (survey [1]): explicit confirmation asks a yes/no question about the recognised content; implicit confirmation repeats it inside the next prompt and lets the user object. Explicit confirmation costs a turn per item.
- **Non-understanding recovery strategies** [2]: an empirical study of ten recovery strategies in a deployed spoken dialogue system, including how strategy choice affects recovery and whether a better selection policy can be learned.
- **POMDP dialogue management** [3]: a belief over user goals is maintained from N-best recognition hypotheses, and when to confirm is chosen by an optimised policy; hand-crafted strategies are shown to be special cases.
- **Read-back of safety-related instructions** (ICAO PANS-ATM, Doc 4444) [4]: the recipient reads back safety-related parts of a clearance and the sender checks the read-back.

Approach	When the system confirms	Uses recogniser uncertainty?	Policy	This protocol, by comparison
Explicit / implicit verification [1]	per item, by design	often (confidence thresholds)	hand-designed	explicit only, and only for alias-corrected transcripts
Recovery strategies [2]	on non-understanding	yes	chosen or learned	a single strategy: refuse and ask the operator to repeat
POMDP dialogue [3]	when expected value favours it	yes, N-best + confidence	optimised	top hypothesis only; fixed rules
ATC read-back [4]	every safety-related instruction	human hearing	procedural	read-back ("Did you mean ...?") only after a correction
This document	transcript changed by the alias table; CONFIRM band	no (only the first alternative's text is used; no score)	fixed, deterministic	—

Where this protocol is weaker. A transcript that is misrecognised as a different *valid* command (e.g. the wrong aircraft name spelled correctly) is not confirmed by this layer; protection then rests on the bands of Algorithms 1 and 3. The recogniser's confidence and its other two alternatives (the dashboard requests `maxAlternatives = 3`) are ignored, and the policy is not learned or evaluated on recorded speech. **Where it differs.** The only automatic rewriting is a short, published alias table, every rewrite is read back and needs a spoken "yes", accepting a correction never authorises the command itself (a CONFIRM-band result needs a second, explicit "confirm"), and emergency phrases are exempt so that a read-back never delays a stop or landing.

10. Relation to other algorithms

Front end of Algorithm 2 (emergency short-cut) and of the pipeline Algorithm 3 → Algorithm 1. Browser-side timing of voice-to-motion is logged separately (`/api/hu/latency`, `logs/hu_voice_to_motion.jsonl`) without influencing decisions.

11. References

1. M. F. McTear, "Spoken dialogue technology: enabling the conversational user interface," *ACM Computing Surveys*, 34(1):90–169, 2002. `https://doi.org/10.1145/505282.505285`
2. D. Bohus and A. I. Rudnicky, "Sorry, I Didn't Catch That! – An Investigation of Non-understanding Errors and Recovery Strategies," *Proc. 6th SIGdial Workshop on Discourse and Dialogue*, Lisbon, 2005, pp. 128–143. `https://aclanthology.org/2005.sigdial-1.14`
3. J. D. Williams and S. Young, "Partially observable Markov decision processes for spoken dialog systems," *Computer Speech & Language*, 21(2):393–422, 2007. `https://doi.org/10.1016/j.csl.2006.06.008`
4. ICAO, *Procedures for Air Navigation Services — Air Traffic Management* (PANS-ATM), Doc 4444, 16th ed., 2016 (read-back of clearances and safety-related information). Copy published by the Swiss Federal Office of Civil Aviation: `https://www.bazl.admin.ch/dam/it/sd-web/jMIWMg9YgaoW/icao_doc_4444_airtrafficmanagement.pdf`

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.